

FutureProof Financial

Platform & AI Strategy

What we're building, and how we build it

FutureProof has built a new kind of mortgage — one that pays the homeowner — and a working system to demonstrate it. This paper is the practical plan for the platform that takes it to every market: **what we are building, the technology we have chosen and why, how the pieces fit together, and how we develop, test, deploy and run it.** It is deliberately concrete.

June 2026

1. What we're building

One platform, owned by FutureProof, that runs the product in every market.

Licensed lenders run on it — one or several per market — each white-labelled, under its own licence. We own the engine; the lenders drive. In plain terms:

- **One product brain.** Pricing, the actuarial models and the business rules live in one place, owned by us, and are identical in every market. A market configures within bounds we set; no lender can fork the product.
- **Many lenders, cleanly separated.** Each lender is a branded, isolated tenant with its own licence and its own ring-fenced data — several can operate in one market. Adding a lender (or a market) means standing up a tenant and a licence, not rebuilding the product.
- **Run by a small team and five AI agents.** A standing team of agents does the repeatable work in every market at once, with a person on every consequential decision.
- **Market-agnostic by design.** Australia is the live build. The platform is built so the next market — whichever it is — is a configuration-and-licence exercise, measured in months.

The rest of this paper is the “how”: the stack and why (Section 2), the system and how it fits together (Section 3), and the way we work — build, test, deploy, run (Section 4).

2. The stack, and why

Why Rails

We build on Ruby on Rails. The reasons compound for a small team building with AI:

- **It is token-efficient — which matters because we build with AI.** Two decades of Rails code and documentation sit in every model's training data, and Rails' "convention over configuration" means there is usually one idiomatic way to do a thing — so our AI agents' first attempt is more often correct and consistent. Rails is also concise: little boilerplate, so the agents generate less code, hold more of the system in context at once, and we have less to review. Less code, more correctness, faster iterations.
- **A small team ships a lot.** Batteries are included — database, background jobs, email, authentication and security defaults all in the box (Rails 8 bundles its own job queue and auth) — and with Stimulus/Hotwire the same framework renders the interface. No separate frontend stack or team: one language, one codebase, one deploy.
- **It fits the architecture we chose.** We are building a modular monolith with a separate database per lender. Rails has had first-class support for multiple databases and per-request connection switching since version 6 — the exact mechanism our tenant isolation needs. The framework and the architecture pull the same way.
- **Boring, stable and secure.** Rails is twenty years old, prizes backwards-compatibility, and does not churn the way the JavaScript ecosystem does — the right trait for a regulated platform meant to run for years. It is secure by default, with built-in protection against the common web vulnerabilities (cross-site request forgery, SQL injection, cross-site scripting) and encrypted credentials.
- **Proven at any scale we will reach.** Shopify, GitHub and GitLab run enormous Rails monoliths today, and Basecamp — which created Rails — runs its products on it. It carries companies from the first commit to global scale; we will not out-grow it before we need to.

The rest of the stack

Layer	Choice	Why
Framework	Ruby on Rails	Token-efficient, battle-tested, fast for a small team (above)
Database	PostgreSQL	Rock-solid; a database per lender gives clean isolation, and per-market hosting keeps data resident
Interface	Server-rendered + Stimulus	Logic stays on the server; the browser does presentation only — simpler and safer
Styling	Our own CSS design system	Full control of every market's brand; no third-party framework to fight or outgrow
Background work	Solid Queue	Jobs run on the database we already have — no extra moving parts to operate
Hosting	Fly.io	Simple, multi-region deploys close to each market; scales when we need it

What we deliberately don't do (yet)

Keeping the system small is a feature. We do not add Kafka, microservices or a sprawling cloud estate before scale forces them, and we do not claim certifications (SOC 2, ISO 27001) before we have earned them. Each piece of weight is added only when it pays for itself. This is the mainstream engineering view in 2026, and it is what lets a small team build quickly and cheaply.

3. The system, and how it fits together

Six parts. The chart reads top to bottom — a customer enters at the top, a funded mortgage sits at the bottom.



Agent	Role
Akane	Customer acquisition — first contact, qualification, guiding the application
Misato	Customer communications and service after onboarding
Rie	Back-office operations
Yumi	Investment account management
Motoko	Engineering and ops — builds and runs the other four

The line our AI stays behind. There is a hard regulatory line between general guidance and personal financial advice. Our agents inform, explain and run the operation; they never tell a customer this product suits *them* and they should take it — that is personal advice, which requires a licence and, in the UK, a qualified human adviser. The AI makes our people faster; it does not replace the regulated judgement.

4. How we work

The same disciplined loop for every component, from how it looks to how it runs. Our demonstration system already works to this standard — 382 automated tests, zero failures — and the platform inherits it.

Design & UX

- **One design system, themed per lender.** A single custom-built component library — buttons, cards, forms, alerts — owned by us. Each lender gets its own brand skin (colour, logo, type) over the same proven patterns: consistency for us, distinctiveness for them, and no design rework per market.
- **Designed for the people who use it.** Our customers are homeowners, often retired and not digital natives, making a major financial decision. The interface is plain-language, uncluttered and accessible — large legible type, clear steps, no jargon — explaining a genuinely new product simply, never in the language of debt.
- **The interface respects the advice wall.** It informs and explains — what the income is, how it works, what happens at the end — but never pressures or tells a customer “you should.” For a regulated product, clarity and honesty are design requirements, not afterthoughts.

Development

We build a modular monolith one component at a time, a small team working alongside AI coding agents. Each component follows one loop:

1. **Refresh the prompt** — pin down what the component does, its inputs and outputs, its definition of done, and the rules it must respect, so the AI builds from accurate context.
2. **Collect test data** — gather realistic data (customers, properties, quotes, market configs) that becomes the fixtures the tests run on.
3. **Build** — the smallest slice that meets the definition of done; business logic on the server, the browser for presentation only.
4. **Test, review, deploy** — covered below. Then move to the next component.

Testing

- **Test-first and integration-first.** We write a test that drives the real path, then make it pass; we verify in a real browser against the real URL, not just in isolation.
- **Nothing ships on a red suite.** The full automated test suite must pass before a change moves on.
- **Security and quality gates on every change:** an automated security scan (Brakeman), a style check (RuboCop), and a content-security check — run before every commit.

Deployment

- **One safe path to production.** Deploys go to Fly.io with a fixed pre-flight — confirm the account, confirm the app, confirm status, then deploy.
- **Released behind configuration, per lender and per market.** A lender or a market can be switched on, or a feature rolled out, without touching another.
- **Reversible, idempotent database changes.** Migrations can always be rolled back; we never destructively touch data.

Change control — humans and AI through one door

- **Every change is a proposal; a person approves it.** Whether a change comes from an engineer, an AI coding agent, or a business user describing what they want in plain language, it enters the same pipeline: a reviewable proposal, with its potential impact on data or functionality declared up front and recorded permanently. Nothing reaches the product without human sign-off.
- **Business people have a direct, safe line to the product.** From the admin they can propose changes to the AI agents' instructions or describe a change in plain language — an AI drafts the implementation, automated tests check it, and a person reviews and approves. Responsibility with safety, no technical skills required.
- **AI autonomy is earned, not assumed.** AI-authored changes start as drafts a person always reviews. Only when the track record shows consistent, unedited approvals does the lowest-risk category move faster — and never to production directly. Risky areas (the financial model, security, deployment) always require human review.
- **Changes are tested before customers see them.** Approved work lands on a staging copy of the platform first; promotion to production is a deliberate human step.

Management

- **Run by the agents, governed by people.** Operations run on the five agents, with a person on every consequential decision; Motoko (engineering/ops) builds and runs the other four.
- **The signal we watch is Investment Health** — the share of accounts in good standing. (An EPM customer never pays, so there is no concept of arrears.)
- **Everything is auditable.** Every agent decision is logged and explainable; model and compliance governance is owned centrally; each lender's data is isolated and stays in its market's jurisdiction.

In short: a proven, token-efficient stack, owned in the centre and isolated at the edge, with a build-test-deploy-run loop a small team can sustain. That makes “one engine, every market” a plan, not an ambition.